



# **PHP-MySQL**

## **(Pages Web Dynamiques)**

# Introduction(1)

## ■ Internet et les pages web

- HTML : conception de pages destinées à être publiées sur Internet
- Page html : contient le texte à afficher et des instructions de mise en page
- HTML est un langage de description de page et non pas un langage de programmation
  - pas d'instructions de calcul ou pour faire des traitements suivant des conditions
- Des sites de plus en plus riches en informations
  - Nécessité croissante d'améliorer le contenu de sites
  - Mises à jour manuelles trop complexes
    - Pourquoi ne pas automatiser les mises à jour ?

# Introduction(2)

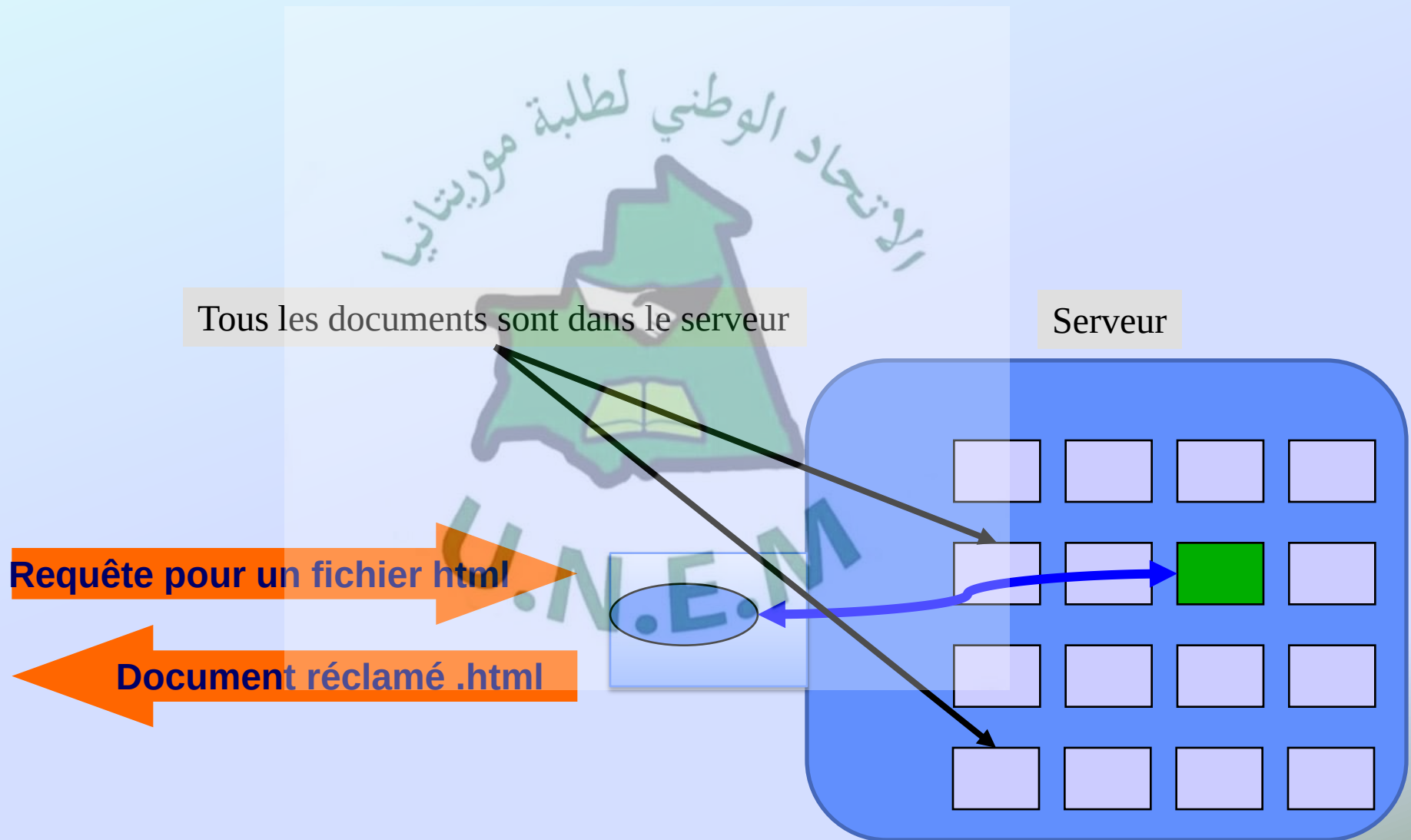
## ■ Pages web statiques : fonctionnement

- Leurs contenus ne changent ni en fonction du demandeur ni en fonction d'autres paramètres éventuellement inclus dans la requête adressée au serveur. Toujours le même résultat.
  - Rôle du serveur : localiser le fichier correspondant au document demandé et répond au navigateur en lui envoyant le contenu de ce fichier

## ■ Pages web statiques : limites

- Besoin de réponses spécifiques : passage de pages statiques à pages dynamiques

# Introduction(3)



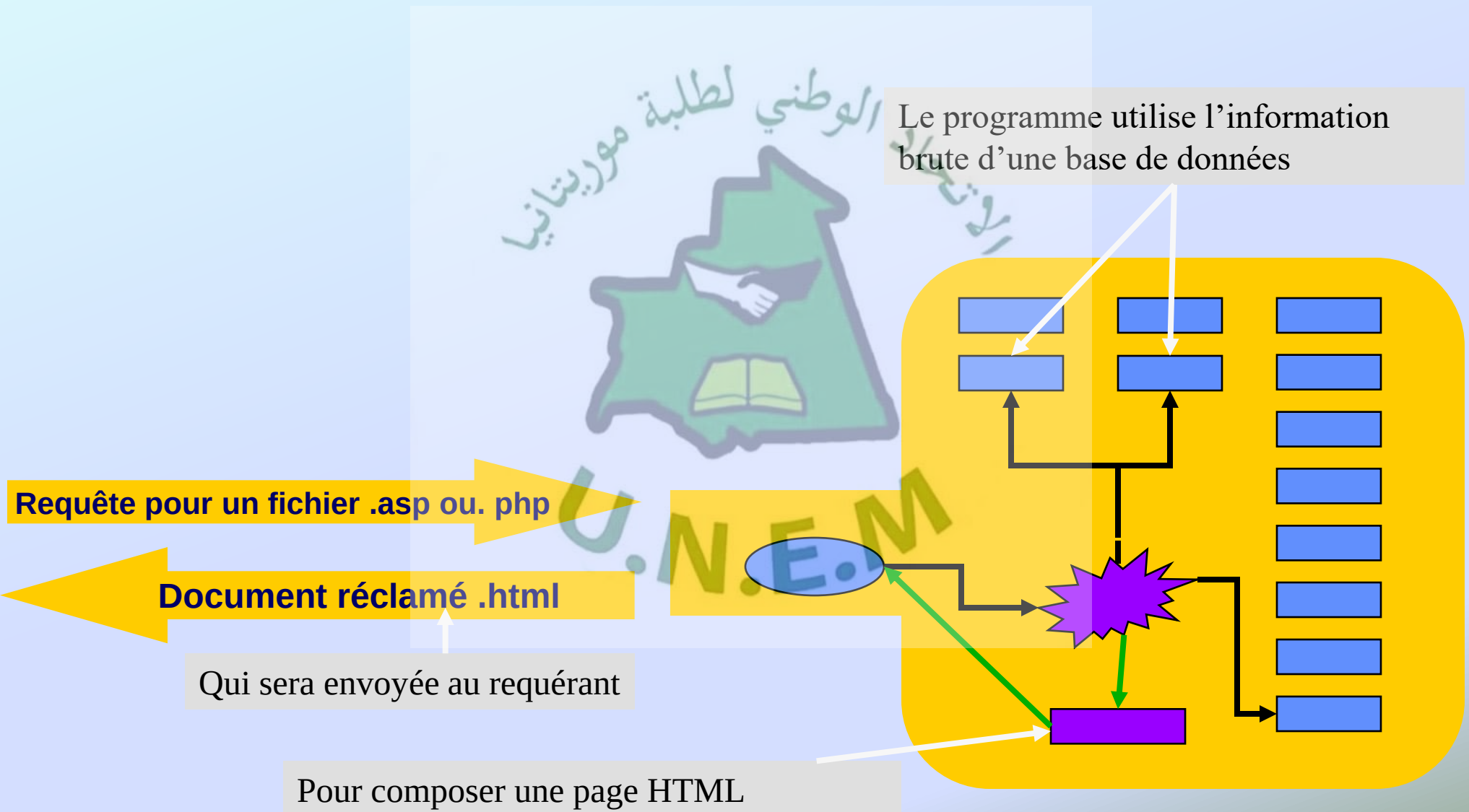


# Introduction(4)

## ■ Les langages de script-serveur : Définition

- Un langage de script -serveur est :
  - un programme stocké sur un serveur et exécuté par celui-ci,
  - qui passe en revue les lignes d'un fichier source pour en modifier une partie du contenu,
  - avant de renvoyer à l'appelant ( un navigateur par exemple) le résultat du traitement.
- La tâche d'interprétation des ordres à exécuter est déléguée à un composant, souvent appelé moteur,
  - installé sur le serveur,
  - qui est doté d'une API et d'un fonctionnement identique quel que soit la plate-forme utilisée pour gérer le serveur

# Introduction(5)



# Introduction(6)

## ■ Pages web dynamiques côté serveur ou côté client

- **Langage côté client** : traité par la machine qui accueille le logiciel de navigation.
  - Ses résultats peuvent varier en fonction de plate-forme utilisée. Un programme en JavaScript pourra fonctionner sous Netscape et poser problème sous Internet explorer.
  - Les résultats peuvent être différents suivant la machine (PC, Mac)
  - Nécessité de tests importants
  - Ne permettent pas de masquer les sources du programme
  - Sont indépendants du serveur et donc de l'hébergement

# Introduction(7)

## ■ Pages web dynamiques côté serveur ou côté client

- Langage côté serveur : le travail d'interprétation du programme est réalisé par le serveur
  - Sont indépendants de la machine et du logiciel de navigation utilisés pour la consultation.
  - Sont compatibles avec tous les navigateurs et toutes leurs versions.
  - Permettent de masquer les sources de ses programmes
  - Nécessitent de recharger la page chaque fois que celle-ci est modifiée.
- Pages côté serveur et côté client :
  - Script côté client pour des calculs et des traitement simples
  - Scripts côté serveur pour des calculs, des traitements et des mises à jours plus conséquents

# Introduction(8)

## ■ Les langages de création de pages web dynamiques côté serveur

### ● ASP

- Basé sur des scripts écrits en VBscript, Jscript ou Javascript.
- Largement répandu,
- Facilité de mise en œuvre
- Intimement liée à l'environnement Windows NT/2000 et au serveur IIS (Internet Information Server) de Microsoft.
  - L'environnement Microsoft est nécessaire



# Introduction(9)

## ■ Les langages de création de pages web dynamiques côté serveur

### ● JSP

- Constitue la réponse de Sun aux ASP de Microsoft
- Utilisation de Java
  - Au départ simple extension du langage Java
  - Est devenu un véritable langage de développement web
- Possède une interface de qualité
- Lenteur relative



# Introduction(10)

- Les langages de création de page web dynamiques côté serveur
  - PHP
    - Connaît un succès toujours croissant sur le Web et se positionne comme un rival important pour ASP
    - Combiné avec le serveur Web Apache et la base de données MySQL, PHP offre une solution particulièrement robuste, stable et efficace
    - Gratuité : Tous les logiciels sont issus du monde des logiciels libres (Open Source).

# Un peu d'histoire

## ■ Histoire et Origine

- PHP : Hypertext PreProcessor
- Première version de PHP a été mis au point au début d'automne par Rasmus Lerdorf en 1994
  - Version appelée à l'époque Personal Home Pages
  - Pour conserver la trace des utilisateurs venant consulter son CV sur son site, grâce à l'accès à une base de données par l'intermédiaire de requêtes SQL
- La version 3.0 de PHP fut disponible le 6 juin 1998
- A la fin de l'année 1999, une version bêta de PHP, baptisée PHP4 est apparue
- En 2001 cinq millions de domaines utilisent PHP
  - trois fois plus que l'année 2000

# PHP : C'est QUOI ?

## ■ Définition

- Un langage de scripts serveur permettant la création d'applications Web
- Indépendant de la plate-forme utilisée par le client puisqu'il est exécuté côté serveur et non côté client.
- La syntaxe du langage provient de celles du langage C, du Perl et de Java.
- Ses principaux atouts sont:
  - La gratuité et la disponibilité du code source (PHP est distribué sous licence GNU GPL)
  - La simplicité d'écriture de scripts
  - La possibilité d'inclure le script PHP au sein d'une page HTML
  - La simplicité d'interfaçage avec des bases de données
  - L'intégration au sein de nombreux serveurs web (Apache, Microsoft IIS, ...)

# Intégration PHP et HTML (1)

## ■ Principe

- Les scripts PHP sont généralement intégrés dans le code d'un document HTML
- L'intégration nécessite l'utilisation de balises

➤ Avec le style PHP : `<?php` ligne de code PHP `?>`

➤ avec le style XML : `<? ligne de code PHP ?>`

➤ avec le style JavaScript :

`<script language=« php »> ligne de code PHP </script>`

# Intégration PHP et HTML (2)

## ■ Forme d'une page PHP

- Intégration directe

```
<?php
//ligne de code PHP
?>
<html>
<head> <title> Mon script PHP
</title> </head>
<body>
//ligne de code HTML
<?php
//ligne de code PHP
?>
//ligne de code HTML
...
</body> </html>
```

# Intégration PHP et HTML (3)

- **Forme d'une page PHP**
  - Inclure un fichier PHP dans un fichier HTML : include()

Fichier Principal

```
<html>
<head>
<title> Fichier d'appel </title>
</head>
<body>
<?php
$salut = " BONJOUR" ;
include "information.php" ;
?>
</body>
</html>
```

Fichier à inclure : information.php

```
<?php
$chaine = $salut. " , C'est PHP " ;
Print "<table border= '3'>
      <tr><td width='100%'>
          <h2> $chaine</h2>
      </td></tr>
      </table>";
?>
```



# Intégration PHP et HTML (4)

## ■ Envoi du code HTML par PHP

- La fonction echo : `echo Expression;`
  - `echo "Chaine de caracteres";`
  - `echo (1+2)*87;`
- La fonction print : `print(expression);`
  - `Print "Chaine de caracteres";`
  - `print ((1+2)*87);`
- La fonction printf : `printf (chaîne formatée);`
  - `printf ("Le périmètre du cercle est %d",$Perimetre);`

# Syntaxe de base : *Introduction*

## ■ Typologie

- Toute instruction se termine par un point-virgule
- Sensible à la casse
  - Sauf par rapport aux fonctions

## ■ Les commentaires

- `/* Voici un commentaire! */`
- `//` un commentaire sur une ligne

# Syntaxe de base : *Les constantes*

## ■ Les constantes

- **Define**("nom\_constant", valeur\_constant );

- `define ("ma_const", "Vive PHP") ;`
- `define ("an", 2018) ;`
- `Print an; print ma_const ;`

- Les constantes prédéfinies

- `NULL`
- `__FILE__`
- `__LINE__`
- `PHP_VERSION`
- `PHP_OS`
- `TRUE` et `FALSE`
- `E_ERROR`

# Syntaxe de base : *Les variables* (1)

## ■ Principe

- Commencent par le caractère \$
- N'ont pas besoin d'être déclarées

## ■ Fonctions de vérifications de variables

- Doubleval(), intval(), strval(),
- is\_bool(), is\_double(), is\_float(), is\_int(), is\_integer(), is\_long(), is\_array(), is\_object(), is\_real(), is\_numeric(), is\_string()
- empty(), gettype(), settype(), Isset(), unset()

## ■ Affectation par valeur et par référence

- Affectation par valeur :  $\$b = \$a$
- Affectation par (référence) variable :  $\$c = \&\$a$

# Syntaxe de base : *Les variables*(2)

## ■ Visibilité des variables

- Variable locale
  - Visible uniquement à l'intérieur d'un contexte d'utilisation
- Variable globale
  - Visible dans tout le script
  - Utilisation de l'instruction `global()` dans des contextes locales

```
<?php
$vr = 100;
function test(){
    global $vr;
    $x = 3;
    return $vr + $x;
}
$resultat = test();
if ($resultat) echo $resultat; else echo " erreur ";
?>
```

# Syntaxe de base : *Les variables*(3)

## ■ Les variables dynamiques

- Permettent d'affecter un nom différent à une autre variable

```
$nom_variable = 'nom_var';  
$$nom_variable = 22; // équivaut à $nom_var = 22;
```

- Les variables tableaux sont également capables de supporter les noms dynamiques

```
$nom_variable = array("val0", "val1", ..., "valN");  
${$nom_variable[0]} = valeur; //$val0 = valeur;  
$nom_var = array();  
$nom_variable = "nom_var";  
${$nom_variable}[0] = valeur; //$nom_var[0] = valeur;
```

- Les accolades servent aussi à éviter toute confusion lors du rendu d'une variable dynamique

```
echo "Nom : $nom_variable - Valeur : ${$nom_variable}";  
// équivaut à echo "Nom : $nom_variable - Valeur :  
$nom_var";
```



# Syntaxe de base : *Les variables* (4)

## ■ Variables prédéfinies

- Les variables d'environnement dépendant du client

| Variable                                       | Description                                                                                     |
|------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <code>\$_SERVER["HTTP_HOST"]</code>            | Nom d'hôte de la machine du client (associée à l'adresse IP)                                    |
| <code>\$_SERVER["HTTP_REFERER"]</code>         | URL de la page qui a appelé le script PHP                                                       |
| <code>\$_SERVER["HTTP_ACCEPT_LANGUAGE"]</code> | Langue utilisée par le serveur (par défaut en-us)                                               |
| <code>\$_SERVER["HTTP_ACCEPT"]</code>          | Types MIME reconnus par le serveur (séparés par des virgules)                                   |
| <code>\$_SERVER["CONTENT_TYPE"]</code>         | Type de données contenu présent dans le corps de la requête. Il s'agit du type MIME des données |
| <code>\$_SERVER["REMOTE_ADDR"]</code>          | L'adresse IP du client appelant le script CGI                                                   |
| <code>\$_SERVER["PHP_SELF"]</code>             | Nom du script PHP                                                                               |

# Syntaxe de base : *Les variables* (5)

## ■ Variables prédéfinies

- Les variables d'environnement dépendant du serveur

| Variable                                  | Description                                                               |
|-------------------------------------------|---------------------------------------------------------------------------|
| <code>\$_SERVER["SERVER_NAME"]</code>     | Le nom du serveur                                                         |
| <code>\$_SERVER["SERVER_ADDR"]</code>     | Adresse IP du serveur                                                     |
| <code>\$_SERVER["SERVER_PROTOCOL"]</code> | Nom et version du protocole utilisé pour envoyer la requête au script PHP |
| <code>\$_SERVER["DATE_GMT"]</code>        | Date actuelle au format GMT                                               |
| <code>\$_SERVER["DATE_LOCAL"]</code>      | Date actuelle au format local                                             |
| <code>\$_SERVER["DOCUMENT_ROOT"]</code>   | Racine des documents Web sur le serveur                                   |

# Syntaxe de base : *Les variables (6)*

## ■ Variables prédéfinies

### ➤ Affichage des variables d'environnement

- la fonction `phpinfo()`
  - `<?php phpinfo(); ?>`
  - `phpinfo(constante);`

|                    |                                                         |
|--------------------|---------------------------------------------------------|
| INFO_CONFIGURATION | affiche les informations de configuration.              |
| INFO_CREDITS       | affiche les informations sur les auteurs du module PHP  |
| INFO_ENVIRONMENT   | affiche les variables d'environnement.                  |
| INFO_GENERAL       | affiche les informations sur la version de PHP.         |
| INFO_LICENSE       | affiche la licence GNU Public                           |
| INFO_MODULES       | affiche les informations sur les modules associés à PHP |
| INFO_VARIABLES     | affiche les variables PHP prédéfinies.                  |

- la fonction `getenv()`
  - `<? echo getenv("HTTP_USER_AGENT"); ?>`

# Syntaxe de base : *Les types de données*

## ■ Principe

- Pas besoin d'affecter un type à une variable avant de l'utiliser
  - La même variable peut **changer** de type en cours de script
  - Les variables issues de l'envoi des données d'un formulaire sont du type string

## ■ Les différents types de données

- Les entiers : le type **Integer**
- Les flottants : le type **Double**
- Les tableaux : le type **array**
- Les chaînes de caractères : le type **string**
- Les objets

# Syntaxe de base : *Les types de données (2)*

## ■ Le transtypage

- La fonction **settype()** permet de convertir le type auquel appartient une variable

```
<?php $nbre=10; echo gettype($nbre);  
        Settype($nbre, "double");  
        Echo '<br>la variable $nbre est devenu de type ' , gettype($nbre); ?>
```

- Transtypage explicite : le cast

➤ (int), (integer) ; (real), (double), (float); (string); (array); (object)

```
<?php $var=" 100 MRU ";  
        Echo ' pour commencer, le type de la variable $var est ', gettype($var) ;  
        $var =(double) $var;  
        Echo ' <br> Après le cast, le type de la variable $var est ', gettype($var);  
        Echo "<br> et a la valeur $var ";  ?>
```



# Syntaxe de base : *Les chaînes de caractères*(1)

## ■ Principe

- Peuvent être constituées de n'importe quel caractère alphanumérique et de ponctuation, y compris les caractères spéciaux

`\tLa nouvelle monnaie unique, l' €uro, est enfin là...\n\r`

- Une chaîne de caractères doit être toujours entourée par des guillemets simples (') ou doubles (")

" Ceci est une chaîne de caractères valide."

'Ceci est une chaîne de caractères valide.'

"Ceci est une chaîne de caractères invalide.'

- Des caractères spéciaux à insérer directement dans le texte, permettent de créer directement certains effets comme des césures de lignes

| Car               | Code ASCII | Code hex | Description                      |
|-------------------|------------|----------|----------------------------------|
| <code>\car</code> |            |          | échappe un caractère spécifique. |
| <code>" "</code>  | 32         | 0x20     | un espace simple.                |
| <code>\t</code>   | 9          | 0x09     | tabulation horizontale           |
| <code>\n</code>   | 13         | 0x0D     | nouvelle ligne                   |
| <code>\r</code>   | 10         | 0x0A     | retour à chariot                 |
| <code>\0</code>   | 0          | 0x00     | caractère NUL                    |
| <code>\v</code>   | 11         | 0x0B     | tabulation verticale             |



# Syntaxe de base : *Les chaînes de caractères*(2)

## ■ Quelques fonctions de manipulation

**chaîne\_result = addCSlashes(chaîne, liste\_caractères);**

ajoute des slashes dans une chaîne

**chaîne\_result = addSlashes(chaîne);**

ajoute un slash devant tous les caractères spéciaux.

**chaîne\_result = trim(chaîne);**

supprime les espaces blancs au début et en fin de chaîne.

**caractère = chr(nombre);**

retourne un caractère en mode ASCII

**chaîne\_result = crypt(chaîne [, chaîne\_code])**

code une chaîne avec une base de codage.

**\$tableau = explode(délimiteur, chaîne);**

scinde une chaîne en fragments à l'aide d'un délimiteur et retourne un tableau.

**\$ chaîne\_result = implode(délimiteur, \$tableau);**

# Syntaxe de base : *les opérateurs* (1)

## ■ Les opérateurs

- les opérateurs de calcul
- les opérateurs d'assignation
- les opérateurs d'incrément
- les opérateurs de comparaison
- les opérateurs logiques
- les opérateurs bit-à-bit
- les opérateurs de rotation de bit

# Syntaxe de base : *Les opérateurs*(2)

## ■ Les opérateurs de calcul

| Opérateur | Dénomination                | Effet                             | Exemple | Résultat                             |
|-----------|-----------------------------|-----------------------------------|---------|--------------------------------------|
| +         | opérateur d'addition        | Ajoute deux valeurs               | $\$x+3$ | 10                                   |
| -         | opérateur de soustraction   | Soustrait deux valeurs            | $\$x-3$ | 4                                    |
| *         | opérateur de multiplication | Multiplie deux valeurs            | $\$x*3$ | 21                                   |
| /         | plus: opérateur de division | Divise deux valeurs               | $\$x/3$ | 2.3333333                            |
| =         | opérateur d'affectation     | Affecte une valeur à une variable | $\$x=3$ | Met la valeur 3 dans la variable \$x |

# Syntaxe de base : *Les opérateurs*(3)

## ■ Les opérateurs d'assignation

| Opérateur | Effet                                                                             |
|-----------|-----------------------------------------------------------------------------------|
| +=        | addition deux valeurs et stocke le résultat dans la variable (à gauche)           |
| -=        | soustrait deux valeurs et stocke le résultat dans la variable                     |
| *=        | multiplie deux valeurs et stocke le résultat dans la variable                     |
| /=        | divise deux valeurs et stocke le résultat dans la variable                        |
| %=        | donne le reste de la division deux valeurs et stocke le résultat dans la variable |
| =         | Effectue un OU logique entre deux valeurs et stocke le résultat dans la variable  |
| ^=        | Effectue un OU exclusif entre deux valeurs et stocke le résultat dans la variable |
| &=        | Effectue un Et logique entre deux valeurs et stocke le résultat dans la variable  |
| .=        | Concatène deux chaînes et stocke le résultat dans la variable                     |

# Syntaxe de base : *Les opérateurs*(4)

## ■ Les opérateurs d'incrémentation

| Opérateur | Dénomination     | Effet                            | Syntaxe | Résultat (avec x valant 7) |
|-----------|------------------|----------------------------------|---------|----------------------------|
| ++        | Incrémentatation | Augmente d'une unité la variable | \$x++   | 8                          |
| --        | Décrémentatation | Diminue d'une unité la variable  | \$x--   | 6                          |

## ■ Les opérateurs de comparaison

| Opérateur | Dénomination                     | Effet                                                           | Exemple | Résultat                                             |
|-----------|----------------------------------|-----------------------------------------------------------------|---------|------------------------------------------------------|
| ==        | opérateur d'égalité              | Compare deux valeurs et vérifie leur égalité                    | \$x==3  | Retourne 1 si \$X est égal à 3, sinon 0              |
| <         | opérateur d'infériorité stricte  | Vérifie qu'une variable est strictement inférieure à une valeur | \$x<3   | Retourne 1 si \$X est inférieur à 3, sinon 0         |
| <=        | opérateur d'infériorité          | Vérifie qu'une variable est inférieure ou égale à une valeur    | \$x<=3  | Retourne 1 si \$X est inférieur à 3, sinon 0         |
| >         | opérateur de supériorité stricte | Vérifie qu'une variable est strictement supérieure à une valeur | \$x>3   | Retourne 1 si \$X est supérieur à 3, sinon 0         |
| >=        | opérateur de supériorité         | Vérifie qu'une variable est supérieure ou égale à une valeur    | \$x>=3  | Retourne 1 si \$X est supérieur ou égal à 3, sinon 0 |
| !=        | opérateur de différence          | Vérifie qu'une variable est différente d'une valeur             | \$x!=3  | Retourne 1 si \$X est différent de 3, sinon 0        |

# Syntaxe de base : *Les opérateurs*(5)

## ■ Les opérateurs logiques

| Opérateur | Dénomination | Effet                                                                                                  | Syntaxe                         |
|-----------|--------------|--------------------------------------------------------------------------------------------------------|---------------------------------|
| ou OR     | OU logique   | Vérifie qu'une des conditions est réalisée                                                             | ((condition1)   (condition2))   |
| && ou AND | ET logique   | Vérifie que toutes les conditions sont réalisées                                                       | ((condition1) && (condition2))  |
| XOR       | OU exclusif  | Opposé du OU logique                                                                                   | ((condition1) XOR (condition2)) |
| !         | NON logique  | Inverse l'état d'une variable booléenne (retourne la valeur 1 si la variable vaut 0, 0 si elle vaut 1) | !(condition)                    |

## ■ Les opérateurs bit-à-bit

| Opérateur | Dénomination          | Effet                                                                           | Syntaxe              | Résultat  |
|-----------|-----------------------|---------------------------------------------------------------------------------|----------------------|-----------|
| &         | ET bit-à-bit          | Retourne 1 si les deux bits de même poids sont à 1                              | 9 & 12 (1001 & 1100) | 8 (1000)  |
|           | OU bit-à-bit          | Retourne 1 si l'un ou l'autre des deux bits de même poids est à 1 (ou les deux) | 9   12 (1001   1100) | 13 (1101) |
| ^         | OU exclusif bit-à-bit | Retourne 1 si l'un des deux bits de même poids est à 1 (mais pas les deux)      | 9 ^ 12 (1001 ^ 1100) | 5 (0101)  |
| ~         | Complément (NON)      | Retourne 1 si le bit est à 0 (et inversement)                                   | ~9 (~1001)           | 6 (0110)  |



# Syntaxe de base : *Les opérateurs*(6)

## ■ Les opérateurs de rotation de bit

| Opérateur | Dénomination                                 | Effet                                                                                                                                                                          | Syntaxe            | Résultat  |
|-----------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|-----------|
| <<        | Rotation à gauche                            | Décale les bits vers la gauche (multiplie par 2 à chaque décalage). Les zéros qui sortent à gauche sont perdus, tandis que des zéros sont insérés à droite                     | 6 << 1 (110 << 1)  | 12 (1100) |
| >>        | Rotation à droite avec conservation du signe | Décale les bits vers la droite (divise par 2 à chaque décalage). Les zéros qui sortent à droite sont perdus, tandis que le bit non-nul de poids plus fort est recopié à gauche | 6 >> 1 (0110 >> 1) | 3 (0011)  |

## ■ Autres opérateurs

| Opérateur | Dénomination              | Effet                                             | Syntaxe               | Résultat           |
|-----------|---------------------------|---------------------------------------------------|-----------------------|--------------------|
| .         | Concaténation             | Joint deux chaînes bout à bout                    | "Bonjour"."Au revoir" | "BonjourAu revoir" |
| \$        | Référencement de variable | Permet de définir une variable                    | \$MaVariable = 2;     |                    |
| ->        | Propriété d'un objet      | Permet d'accéder aux données membres d'une classe | \$MonObjet->Propriete |                    |

## Syntaxe de base : *Les opérateurs*(7)

## ■ Les priorités

[illegible]

# Syntaxe de base : *Les instructions conditionnelles(1)*

## ■ L'instruction if

- if (condition réalisée) { liste d'instructions }

## ■ L'instruction if ... Else

- if (condition réalisée) {liste d'instructions}  
else { autre série d'instructions }

## ■ L'instruction if ... elseif ... Else

- if (condition réalisée) {liste d'instructions}  
elseif (autre condition ) {autre série d'instructions }  
else { série d'instructions }

## ■ Opérateur ternaire

- (condition) ? instruction si vrai : instruction si faux;

# Syntaxe de base : *Les instructions conditionnelles(2)*

## ■ L'instruction switch

```
switch (Variable) {  
    case Valeur1: Liste d'instructions break;  
    case Valeur2: Liste d'instructions break;  
    case Valeurs...: Liste d'instructions break;  
    default: Liste d'instructions;  
}
```

# Syntaxe de base : *Les instructions conditionnelles(3)*

## ■ La boucle for

- `for ($i=1; $i<6; $i++) { echo "$i<br>"; }`

## ■ La boucle while

- `While(condition) {bloc d'instructions ;}`
- `While (condition) :Instruction1 ;Instruction2 ;  
.... endwhile ;`

## ■ La boucle do...while

- `Do {bloc d'instructions ;}while(condition) ;`

## ■ La boucle foreach (PHP4)

- `Foreach ($tableau as $valeur) {insts utilisant $valeur ;}`

# Syntaxe de base : *Les fonctions*(1)

## ■ Déclaration et appel d'une fonction

```
function nom_fonction($arg1, $arg2, ...$argn)
{
    déclaration des variables ;
    bloc d'instructions ;
    // fin du corps de la fonction
    return $resultat ;
}
```

## ■ Fonction avec nombre d'arguments inconnu

- **func\_num\_args()** : fournit le nombre d'arguments qui ont été passés lors de l'appel de la fonction
- **func\_get\_arg(\$i)** : retourne la valeur de la variable située à la position \$i dans la liste des arguments passés en paramètres.
  - Ces arguments sont numérotés à partir de 0



# Syntaxe de base : *Les fonctions*(2)

## ■ Fonction avec nombre d'arguments inconnu

```
<?php
function produit()
{
    $nbarg = func_num_args();
    $prod=1;
    // la fonction produit a ici $nbarg arguments
    for ($i=0; $i <$nbarg; $i++)
    {
        $prod *= func_get_arg($i);
    }
    return $prod;
}

echo "le produit est : ", produit (3, 77, 10, 5, 81, 9),
    "<br />";
// affiche le produit est 8 419 950
?>
```

# Syntaxe de base : *Les fonctions*(3)

## ■ Passage de paramètre par référence

- Pour passer une variable par référence, il faut que son nom soit précédé du symbole & (exemple &\$a)

```
<?php
function dire_texte($qui, &$texte){ $texte = "Bienvenue $qui";}
$chaine = "Bonjour ";
dire_texte("cher phpeur", $chaine);
echo $chaine; // affiche "Bienvenue cher phpeur"
?>
```

## ■ L'appel récursif

- PHP admet les appels récursifs de fonctions

# Syntaxe de base : *Les fonctions*(4)

## ■ Appel dynamique de fonctions

- Exécuter une fonction dont le nom n'est pas forcément connu à l'avance par le programmeur du script
- L'appel dynamique d'une fonction s'effectue en suivant le nom d'une variable contenant le nom de la fonction par des parenthèses

```
<?php $datejour = getdate() ; // date actuelle
//récupération des heures et minutes actuelles
$heure = $datejour['hours'] ;      $minute=$datejour['minutes'] ;
function bonjour(){                global $heure;                global $minute;
echo "<b> BONJOUR A VOUS IL EST : ", $heure, " H ", $minute, "</b> <br />" ;}
function bonsoir (){
global $heure ;                global $minute ;
echo "<b> BONSOIR A VOUS IL EST : ", $heure, " H ", $minute , "</ b> <br />" ;}
if ($heure <= 17) {$salut = "bonjour" ; }                else $salut="bonsoir" ;
//appel dynamique de la fonction
$salut() ;      ?>
```

# Syntaxe de base : *Les fonctions*(5)

## ■ Variables locales et variables globales

- variables en PHP : global, local, static
  - toute variable déclarée en dehors d'une fonction est globale
  - utiliser une variable globale dans une fonction nécessite l'instruction de **global** suivie du nom de la variable
  - Pour conserver la valeur acquise par une variable entre deux appels de la même fonction : l'instruction **static**.
- Les variables statiques restent locales à la fonction et ne sont pas réutilisables à l'extérieur.

```
<?php
function cumul ($prix) {
    static $i = 1 ;
    echo "Total des achats $i = ";
    $cumul += $prix; $i++ ;
    return $cumul ; }
echo cumul(175), "<br />";
echo cumul(65), "<br />";
echo cumul(69), "<br />" ;    ?>
```

# Syntaxe de base : *Les tableaux*(1)

## ■ Principe

- Création à l'aide de la fonction **array()**
- Uniquement des tableaux à une dimension
  - Les éléments d'un tableau peuvent pointer vers d'autres tableaux
- Les éléments d'un tableau peuvent appartenir à des types distincts
- L'index d'un tableau en PHP commence de 0
- Pas de limites supérieures pour les tableaux
- La fonction **count()** pour avoir le nombre d'éléments d'un tableau



# Syntaxe de base : *Les tableaux*(2)

## ■ Les tableaux indicés et les tableaux associatifs

### ● Tableau indicé

➤ *Accéder aux éléments par l'intermédiaire de numéros*

```
$tableau = array();
```

```
$tableau[indice] = valeur;
```

```
$jour[3] = "Mercredi";
```

```
$note[0] = 20;
```

```
$tableau = array(valeur0, valeur1, ..., valeurN);
```

```
$jour = array("Dimanche", "Lundi", "Mardi", "Mercredi",  
             "Jeudi", "Vendredi", "Samedi");
```

```
$note = array(20, 15, 12.6, 17, 10, 20, 11, 18, 19);
```

```
$variable = $tableau[indice];
```

```
$JJ = $jour[6]; // affecte "Samedi" à $JJ
```

```
echo $note[1] + $note[5];
```



# Syntaxe de base : *Les tableaux*(3)

## ■ Les tableaux indicés et les tableaux associatifs

- Tableau associatif (ou table de hachage)
  - Les éléments sont référencés par des chaînes de caractères associatives en guise de nom: la clé d'index

```
$tableau = array();
```

```
$tableau["indice"] = valeur;
```

```
$jour["Dimanche"] = 7
```

```
$jour["Mercredi"] = "Le jour des enfants"
```

```
$tableau = array(ind0 => val0, ind1 => val1,..., indN => valN);
```

```
$jour = array("Dimanche" => 1, "Lundi" => 2, "Mardi" => 3, "Mercredi" => 4, "Jeudi" => 5, "Vendredi" => 6, "Samedi" => 7);
```

```
$variable = $tableau["indice"];
```

```
$JJ = $jour["Vendredi"]; //affecte 6 à $JJ
```

```
echo $jour["Lundi"]; //retourne la valeur 2
```

# Syntaxe de base : *Les tableaux*(4)

## ■ Tableaux multidimensionnels

- Pas d'outils pour créer directement des tableaux multidimensionnels
- L'imbrication des tableaux est possible

```
$tab1 = array(Val0, Val1, ..., ValN);
$tab2 = array(Val0, Val1, ..., ValN);
// Création d'un tableau à deux dimensions
$tableau = array($tab1, $tab2);
$mois = array("Janvier", "Février", "Mars", "Avril", "Mai", "Juin",
    "Juillet", "Août", "Septembre", "Octobre", "Novembre",
    "Décembre");
$jour = array("Dimanche", "Lundi", "Mardi", "Mercredi", "Jeudi",
    "Vendredi", "Samedi");
$element_date = array($mois, $jour);

$variable = $tableau[indice][indice];
$MM = $element_date[0][0]; //affecte "Janvier" à $MM
echo $element_date[1][5] . " 7 " . $element_date[0][2] . " 2018";
// retourne "Vendredi 7 Mars 2018"
```

# Syntaxe de base : *Les tableaux*(5)

## ■ Lecture des éléments d'un tableau

### ● Avec une boucle for

```
for ($i=0; $i<count($tab) ; $i++){  
    echo "La cellule n° ". $i . " : " . $tab[$i]. "<br>"; }
```

### ● Avec une boucle while

```
$i=0;  
while (isset($tab[$i])){  
    echo "La cellule n° ". $i . " : " . $tab[$i]. "<br>";  
    $i++;}
```

### ● Avec La boucle foreach

```
$jour = array("Dimanche", "Lundi", "Mardi", "Mercredi", "Jeudi",  
    "Vendredi", "Samedi");  
$i = 0;  
foreach($jour as $JJ) {  
    echo "La cellule n° ". $i . " : " . $JJ . "<br>"; $i++; }
```

# Syntaxe de base : *Les tableaux*(6)

## ■ Lecture des éléments d'un tableau

### ● Parcours d'un tableau associatif

- Réalisable en ajoutant avant la variable *\$valeur*, la clé associée

```
$tableau = array(clé1 => val1, clé2 => val2, ..., cléN => valN);  
foreach($tableau as $clé => $valeur) {  
    echo "Valeur ($clé): $valeur"; }
```

```
$jour = array("Dimanche" => 7, "Lundi" => 1, "Mardi" => 2,  
    "Mercredi" => 3, "Jeudi" => 4, "Vendredi" => 5, "Samedi" =>  
    6);  
foreach($jour as $sJJ => $nJJ) {  
    echo "Le jour de la semaine n° ". $nJJ . " : " . $sJJ . "<br>";  
}
```

# Syntaxe de base : *Les tableaux*(7)

## ■ Fonctions de tri

- Tri selon les valeurs
  - La fonction **sort()** effectue un tri sur les valeurs des éléments d'un tableau selon un critère alphanumérique :selon les codes ASCII :
    - « a » est après « Z » et « 10 » est avant « 9 »)
    - Le tableau initial est modifié et non récupérables dans son ordre original
    - Pour les tableaux associatifs les clés seront perdues et remplacées par un indice créé après le tri et commençant à 0
  - La fonction **rsort()** effectue la même action mais en ordre inverse des codes ASCII.
  - La fonction **asort()** trie également les valeurs selon le critère des codes ASCII, mais en préservant les clés pour les tableaux associatifs
  - La fonction **arsort()** la même action mais en ordre inverse des codes ASCII



# Syntaxe de base : *Les tableaux*(8)

## ■ Fonctions de tri

- Tri sur les clés
- La fonction *krsort()* trie les clés du tableau selon le critère des codes ASCII, et préserve les associations clé / valeur
  - La fonction *krsort()* effectue la même action mais en ordre inverse des codes ASCII

```
<?php
$tab2 = array ("1622"=>"Molière", "1920"=>"Vian ", "1802"=>"Hugo");
krsort ($tab2);
echo "<h3 > Tri sur les clés de \$tab2 </h3>";
foreach ($tab2 as $cle=>$valeur) {
    echo "l'élément a pour clé : $cle et pour valeur : $valeur<br>";
}
?>
```



# La gestion des fichiers avec PHP (1)

## ■ Principe

- PHP prend en charge l'accès au système de fichiers du système d'exploitation du serveur
- Les opérations sur les fichiers concernent la création, l'ouverture, la suppression, la copie, la lecture et l'écriture de fichiers
- Les possibilités d'accès au système de fichiers du serveur sont réglementées par les différents droits d'accès accordés au propriétaire, à son groupe et aux autres utilisateurs
- La communication entre le script PHP et le fichier est repérée par une variable, indiquant l'état du fichier et qui est passée en paramètre aux fonctions spécialisées pour le manipuler

# La gestion des fichiers avec PHP (2)

## ■ Ouverture de fichiers

- La fonction **fopen()** permet d'ouvrir un fichier, que ce soit pour le lire, le créer ou y écrire

**entier fopen(chaine nom du fichier, chaine mode);**

- **mode** : indique le type d'opération qu'il sera possible d'effectuer sur le fichier après ouverture. Il s'agit d'une lettre (en réalité une chaîne de caractères) indiquant l'opération possible:
  - **r** (comme read) indique une ouverture en lecture seulement
  - **w** (comme write) indique une ouverture en écriture seulement (la fonction crée le fichier s'il n'existe pas)
  - **a** (comme append) indique une ouverture en écriture seulement avec ajout du contenu à la fin du fichier (la fonction crée le fichier s'il n'existe pas)
- lorsque le mode est suivie du caractère **+** celui-ci peut être lu et écrit
- le fait de faire suivre le mode par la lettre **b** entre crochets indique que le fichier est traité de façon binaire.

# La gestion des fichiers avec PHP (3)

## ■ Ouverture de fichiers

| Mode | Description                                                                                                                |
|------|----------------------------------------------------------------------------------------------------------------------------|
| r    | ouverture en lecture seulement                                                                                             |
| w    | ouverture en écriture seulement (la fonction crée le fichier s'il n'existe pas)                                            |
| a    | ouverture en écriture seulement avec ajout du contenu à la fin du fichier (la fonction crée le fichier s'il n'existe pas)  |
| r+   | ouverture en lecture et écriture                                                                                           |
| w+   | ouverture en lecture et écriture (la fonction crée le fichier s'il n'existe pas)                                           |
| a+   | ouverture en lecture et écriture avec ajout du contenu à la fin du fichier (la fonction crée le fichier s'il n'existe pas) |

# La gestion des fichiers avec PHP (4)

## ■ Ouverture de fichiers

### ● Exemple

```
$fp = fopen("fichier.txt","r"); //lecture
```

```
$fp = fopen("fichier.txt","w"); //écriture depuis début du  
fichier
```

### ● De plus, la fonction `fopen` permet d'ouvrir des fichiers présents sur le web grâce à leur URL.

- Exemple : un script permettant de récupérer le contenu d'une page d'un site web:

```
<?  
$fp = fopen("http://www.mondomaine.fr","r"); //lecture du  
fichier  
?>
```

# La gestion des fichiers avec PHP (5)

## ■ Ouverture de fichiers

- Il est généralement utile de tester si l'ouverture de fichier s'est bien déroulée ainsi que d'éventuellement stopper le script PHP si cela n'est pas le cas

```
<?php
$fp = fopen("fichier.txt","r");
If($fp==0) {
    echo "Echec de l'ouverture du fichier";
    exit;
}
else {// votre code;}
?>
```

- Un fichier ouvert avec la fonction **fopen()** doit être fermé, à la fin de son utilisation, par la fonction **fclose()** en lui passant en paramètre l'entier retourné par la fonction fopen()



# La gestion des fichiers avec PHP (6)

## ■ Lecture et écriture de fichiers

- Il est possible de lire le contenu d'un fichier et d'y écrire des informations grâce aux fonctions:
  - **fputs()** (ou l'alias **fwrite()**) permet d'écrire une chaîne de caractères dans le fichier. Elle renvoie 0 en cas d'échec, 1 dans le cas contraire
    - booléen **fputs(entier Etat\_du\_fichier, chaine Sortie);**
  - **fgets()** permet de récupérer une ligne du fichier. Elle renvoie 0 en cas d'échec, 1 dans le cas contraire
    - **fgets(entier Etat\_du\_fichier, entier Longueur);**
      - Le paramètre Longueur désigne le nombre de caractères maximum que la fonction est sensée récupérer sur la ligne
      - Pour récupérer l'intégralité du contenu d'un fichier, il faut insérer la fonction **fgets()** dans une boucle while. On utilise la fonction **feof()**, fonction testant la fin du fichier.



# La gestion des fichiers avec PHP (7)

## ■ Lecture et écriture de fichiers

```
<?php
if (!$fp = fopen("fichier.txt", "r"))
    {echo "Echec de l'ouverture du fichier";}
else { $Fichier="";
    while(!feof($fp)) {
        // On récupère une ligne
        $Ligne = fgets($fp,255);
        // On affiche la ligne
        echo $Ligne.'<br>';
        // On stocke l'ensemble des lignes dans une variable
        $Fichier .= $Ligne. "<BR>";
    }
    fclose($fp); // On ferme le fichier
}
?>
```

# La gestion des fichiers avec PHP (8)

## ■ Lecture et écriture de fichiers

- Pour stocker des informations dans le fichier, il faut dans un premier temps ouvrir le fichier en écriture en le créant s'il n'existe pas
  - Deux choix : le mode 'w' et le mode 'a'.

```
<?php
$nom="Ahmed"; $email="ahmed@iscae.mr";
$fp = fopen("fichier.txt","a"); // ouverture du fichier en
    écriture
fputs($fp, "\n"); // on va a la ligne
fputs($fp, $nom."|".$email); // on écrit le nom et email
    dans le fichier
fclose($fp);
?>
```

# La gestion des fichiers avec PHP (9)

## ■ Les tests de fichiers

- **is\_dir()** permet de savoir si le fichier dont le nom est passé en paramètre correspond à un répertoire.
  - La fonction **is\_dir()** renvoie 1 s'il s'agit d'un répertoire, 0 dans le cas contraire

booléen **is\_dir(chaine Nom\_du\_fichier);**

- **is\_executable()** permet de savoir si le fichier dont le nom est passé en paramètre est exécutable.
  - La fonction **is\_executable()** renvoie 1 si le fichier est exécutable, 0 dans le cas contraire

booléen **is\_executable(chaine Nom\_du\_fichier);**

# La gestion des fichiers avec PHP (10)

## ■ Les tests de fichiers

- **is\_link()** permet de savoir si le fichier dont le nom est passé en paramètre correspond à un lien symbolique. La fonction **is\_link()** renvoie 1 s'il s'agit d'un lien symbolique, 0 dans le cas contraire

booléen **is\_link(chaine Nom\_du\_fichier);**

- **is\_file()** permet de savoir si le fichier dont le nom est passé en paramètre ne correspond ni à un répertoire, ni à un lien symbolique.

- La fonction **is\_file()** renvoie 1 s'il s'agit d'un fichier, 0 dans le cas contraire

booléen **is\_file(chaine Nom\_du\_fichier);**

# La gestion des fichiers avec PHP (11)

## ■ D'autres façons de lire et écrire

- La fonction **file()** permet de retourner dans un tableau l'intégralité d'un fichier en mettant chacune de ces lignes dans un élément du tableau

➤ **Tableau** `file(chaine nomdufichier);`

- Exemple : parcourir l'ensemble du tableau afin d'afficher le fichier

```
<?php
$Fichier = "fichier.txt";
if (is_file($Fichier)) {
    if ($TabFich = file($Fichier)) {
        for($i = 0; $i < count($TabFich); $i++)
            echo $TabFich[$i].'<br>';
    }
    else {echo "Le fichier ne peut être lu...<br>";}
}
else {echo "Désolé le fichier n'est pas valide<br> " ;}
?>
```



# La gestion des fichiers avec PHP (12)

## ■ D'autres façons de lire et d'écrire

- La fonction `fpassthru()` permet d'envoyer le contenu d'un fichier dans la fenêtre du navigateur.

`booléen fpassthru(entier etat);`

- Elle permet d'envoyer le contenu du fichier à partir de la position courante dans le fichier.
- Elle n'ouvre pas automatiquement un fichier. Il faut donc l'utiliser avec `fopen()`.
- Il est possible par exemple de lire quelques lignes avec `fgets()`, puis d'envoyer le reste au navigateur.



# La gestion des fichiers avec PHP (13)

**Exemple** : script permettant de parcourir tous les fichiers HTML contenus dans un site à la recherche de MetaTags

```
<?php
function ScanRep($Directory) {
    echo "<b>Parcours</b>: $Directory<br>\n";
    if (is_dir($Directory) && is_readable($Directory)) {
        if ($MyDirectory = opendir($Directory)) {
            while ($Entry = readdir($MyDirectory)) {
                if (is_dir($Directory."/".$Entry)) {
                    if (($Entry != ".") && ($Entry != "..")) {
                        echo "<b>Repertoire</b>: $Directory/$Entry<br>\n";
                        ScanRep($Directory."/".$Entry);
                    }
                }
                else {echo "<b>Fichier</b>: $Directory/$Entry<br>\n";
                    if (eregi("(\\.html)|\\.htm)", $Entry)) {
                        $meta=get_meta_tags($Directory."/".$Entry);
                        foreach($meta as $cle => $valeur) echo $cle." ".$valeur."<br>";
                    }
                }
            }
        }
        closedir($MyDirectory);
    }
    ScanRep(".");
}
?>
```

# La gestion des fichiers avec PHP (14)

## ■ Le téléchargement de fichier

- Le langage PHP4 dispose de plusieurs outils facilitant le téléchargement vers le serveur et la gestion des fichiers provenant d'un client
- Un simple formulaire comportant un champ de type file suffit au téléchargement d'un fichier qui subséquemment, devra être traité par un script PHP adapté

```
<form method="POST" action="traitement.php"
      enctype="multipart/form-data">
  <input type="hidden" name="MAX_FILE_SIZE" value="Taille_Octets">
  <input type="file" name="fichier" size="30"><br>
  <input type="submit" name="telechargement" value="telecharger">
</form>
```

# La gestion des fichiers avec PHP(15)

## ■ Le téléchargement de fichier en PHP4

- Un champ caché doit être présent dans le formulaire afin de spécifier une taille maximum (MAX\_FILE\_SIZE) pour le fichier à télécharger. Cette taille est par défaut égale à deux mégaoctets.
- En PHP 4, le tableau associatif global **\$\_FILES** contient plusieurs informations sur le fichier téléchargé.
  - **`$_FILES['fichier']['name']`** : fournit le nom d'origine du fichier.
  - **`$_FILES['fichier']['type']`** : fournit le type MIME du fichier.
  - **`$_FILES['fichier']['size']`** : fournit la taille en octets du fichier.
  - **`$_FILES['fichier']['tmp_name']`** : fournit le nom temporaire du fichier.

# La gestion des fichiers avec PHP(16)

## ■ Le téléchargement de fichier en PHP3

- PHP 3 fait appel au variable globale ou/et au tableau associatif globale `$HTTP_POST_VARS` à condition que respectivement les options de configuration `register_globals` et `track_vars` soient activées dans le fichier `php.ini`.
  - **`$fichier`** : renvoie le nom temporaire du fichier.
  - **`$fichier_name`** : renvoie le nom d'origine du fichier.
  - **`$fichier_size`** : renvoie la taille en octets du fichier.
  - **`$fichier_type`** : renvoie le type MIME du fichier.
  - **`$HTTP_POST_VARS['fichier']`** : fournit le nom temporaire du fichier.
  - **`$HTTP_POST_VARS['fichier_name']`** : fournit le nom d'origine du fichier.
  - **`$HTTP_POST_VARS['fichier_type']`** : fournit le type MIME du fichier.
  - **`$HTTP_POST_VARS['fichier_size']`** : fournit la taille en octets du fichier.



# La gestion des fichiers avec PHP(17)

## ■ Le téléchargement de fichier

- Par défaut, le fichier envoyé par le client est stocké directement dans le répertoire indiqué par l'option de configuration `upload_tmp_dir` dans le fichier `php.ini`.

**`upload_tmp_dir = c:\PHP\uploadtemp`**

- Plusieurs fonctions spécialisées permettent la validation d'un fichier téléchargé pour son utilisation ultérieure.
  - **La fonction `is_uploaded_file`** indique si le fichier a bien été téléchargé par la méthode HTTP POST.

**`$booleen=is_uploaded_file($_FILES['fichier']['tmp_name']);`**

- **La fonction `move_uploaded_file`** vérifie si le fichier a été téléchargé par la méthode HTTP POST, puis si c'est le cas le déplace vers l'emplacement spécifié.